

Seance intermediaire

Hadoop, HDFS, YARN et MapReduce

Comprendre le socle historique du Big Data avant Apache Spark

Duree cible : 1h30

Format : cours + exercices sur VM Hadoop/YARN

Fil conducteur : passer du fichier local au stockage distribue, puis au calcul distribue.

Pourquoi cette seance intermediaire ?

Faire le pont entre l'integration temps reel et le traitement Big Data batch.

CADRE

Avant

API REST, ActiveMQ, Camel : on a appris a faire circuler des messages et a decoupler les traitements.

Maintenant

Les messages et les logs s'accumulent. Il faut stocker, rejouer et analyser des volumes importants.

Ensuite

Spark va arriver comme moteur moderne. Mais Spark s'appuie souvent sur HDFS et YARN.

Probleme central : que se passe-t-il quand les donnees ne tiennent plus sur une machine et que le calcul doit etre distribue ?

EDA produit les evenements -> Hadoop les stocke et les traite -> Spark accelere et simplifie les traitements

Objectifs et deroule de la seance

En 1h30, l'objectif est de comprendre et manipuler les bases, sans entrer dans l'administration avancee.

OBJECTIFS

1. Comprendre HDFS

Savoir expliquer blocs, replication, NameNode et DataNode.

2. Comprendre YARN

Savoir distinguer stockage distribue et gestion des ressources de calcul.

3. Comprendre MapReduce

Decomposer un probleme en Map, Shuffle et Reduce.

Deroule de la seance

0-10 min

Probleme : volume, disque, memoire, machine unique

10-30 min

HDFS : stockage distribue + mini TP commandes

30-45 min

YARN : ressources, containers, application

45-70 min

MapReduce : modele mental + WordCount

70-85 min

Exercices sur VM Hadoop/YARN

85-90 min

Comparatif MapReduce / Spark

Le probleme : le fichier qui ne rentre plus dans la machine

Le Big Data commence souvent par un probleme tres simple : trop de donnees pour une seule machine.

PROBLEME

Cas fil rouge : ShopFlow

- Chaque commande et chaque consultation produit un log.
- Les evenements sont conserves pour audit, reporting et analyse.
- Les volumes augmentent : jours, mois, annees.
- Le besoin n'est plus seulement temps reel : il faut aussi rejouer et analyser.

1 jour : 20 Go
1 mois : 600 Go
1 an : 7 To

Point de reflexion

Que se passe-t-il si l'on tente de traiter 7 To avec un script Python local ?

Prochaine idee : au lieu de grossir une seule machine, on distribue les donnees et le calcul.

Hadoop en une phrase : distribuer stockage et calcul

Hadoop n'est pas une seule brique : c'est un ecosysteme organise autour de roles distincts.

HADOOP

HDFS

Stocker des fichiers tres volumineux en les decoupant en blocs et en les repliquant sur plusieurs machines.

YARN

Allouer les ressources CPU/RAM du cluster et lancer les applications dans des containers.

MapReduce

Modele de calcul batch historique : Map -> Shuffle -> Reduce.

Stocker

Orchestrer

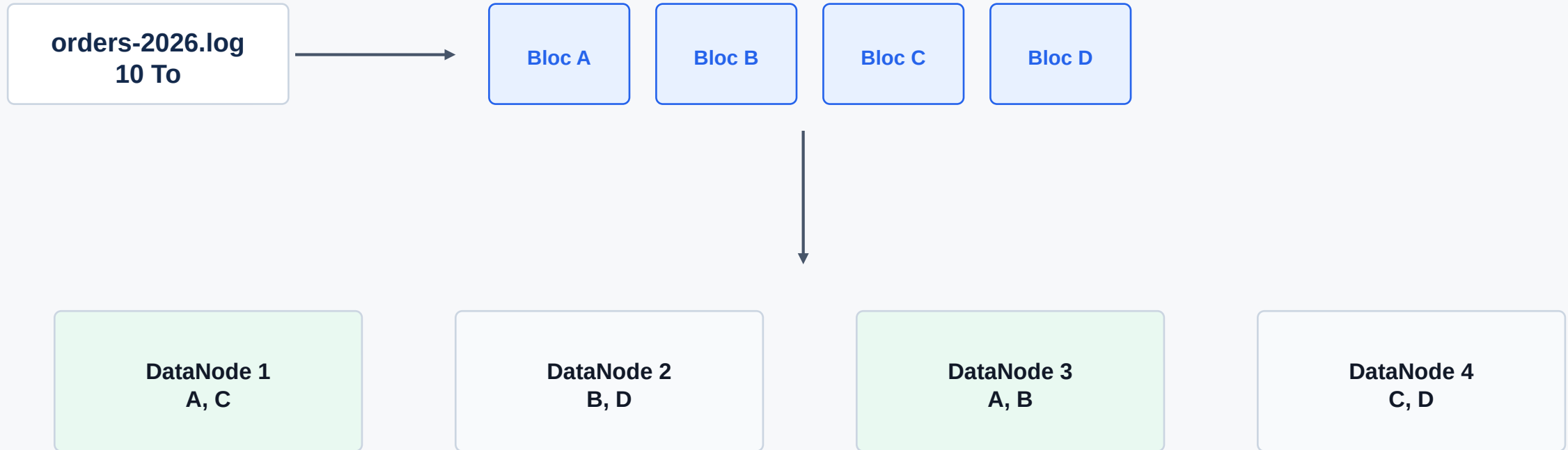
Calculer

A retenir : HDFS garde les donnees, YARN gere les ressources, MapReduce execute le traitement.

HDFS : stocker de gros fichiers sur plusieurs machines

HDFS est pense pour des fichiers volumineux, lus en streaming, avec tolerance aux pannes.

HDFS

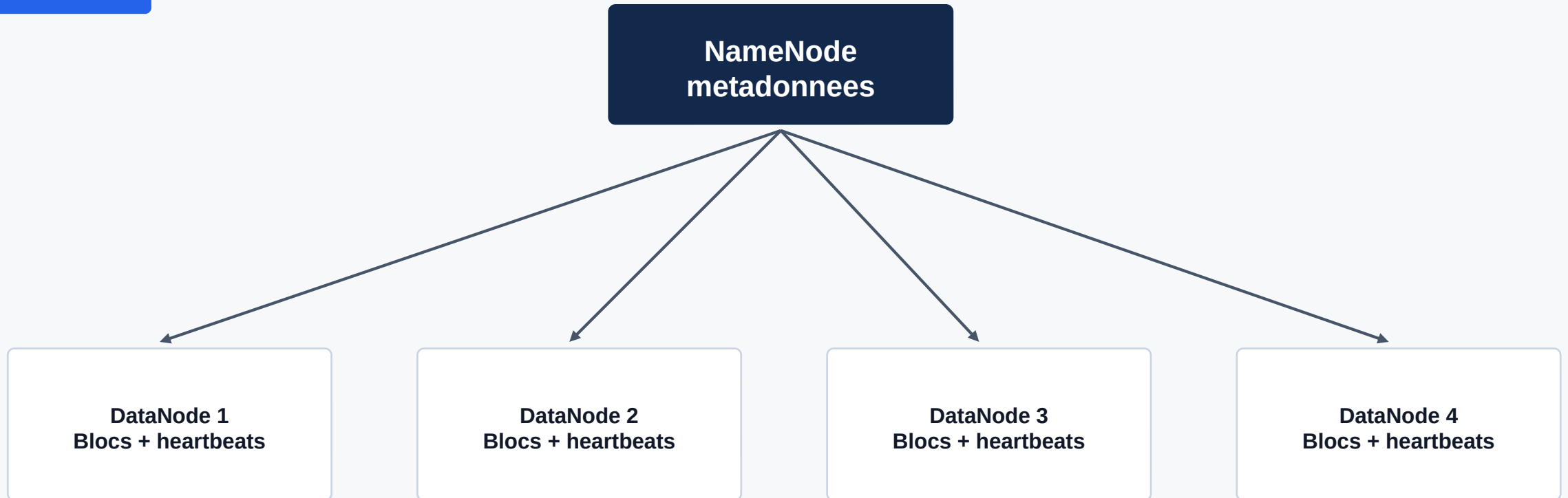


Idee centrale : le fichier n'est plus stocke comme un bloc unique. Il est decoupe, distribue et replique.

Les composants HDFS : NameNode et DataNodes

Le NameNode connaît la carte, les DataNodes portent les blocs.

HDFS



NameNode

Il ne stocke pas les données du fichier. Il stocke les métadonnées : noms de fichiers, blocs, emplacements, droits.

DataNode

Il stocke les blocs réels et informe régulièrement le NameNode de son état.

Replication

Chaque bloc peut exister sur plusieurs DataNodes pour tolérer les pannes.

Lire et ecrire dans HDFS : une logique proche de Linux

Les commandes ressemblent a un shell classique, mais la cible est un systeme distribue.

HDFS TP

```
# Creer un repertoire
hdfs dfs -mkdir -p /shopflow/input

# Envoyer un fichier local vers HDFS
hdfs dfs -put orders.log /shopflow/input/

# Lister et lire
hdfs dfs -ls /shopflow/input
hdfs dfs -cat /shopflow/input/orders.log | head

# Taille et comptage
hdfs dfs -du -h /shopflow/input
hdfs dfs -count /shopflow/input
```

- La commande `hdfs dfs` manipule HDFS, pas le disque local.
- Le chemin `/shopflow/input` est dans HDFS.
- Le fichier local `orders.log` est copie puis distribue par HDFS.
- La commande `-cat` est pratique pour verifier rapidement le contenu.

Piege frequent : `/tmp/orders.log` et `/shopflow/input/orders.log` ne sont pas au meme endroit. Le premier est local, le second est dans HDFS.

Exercice 1 - prise en main HDFS

Objectif : manipuler un fichier dans HDFS et verifier qu'il est accessible.

EXERCICE

Consigne

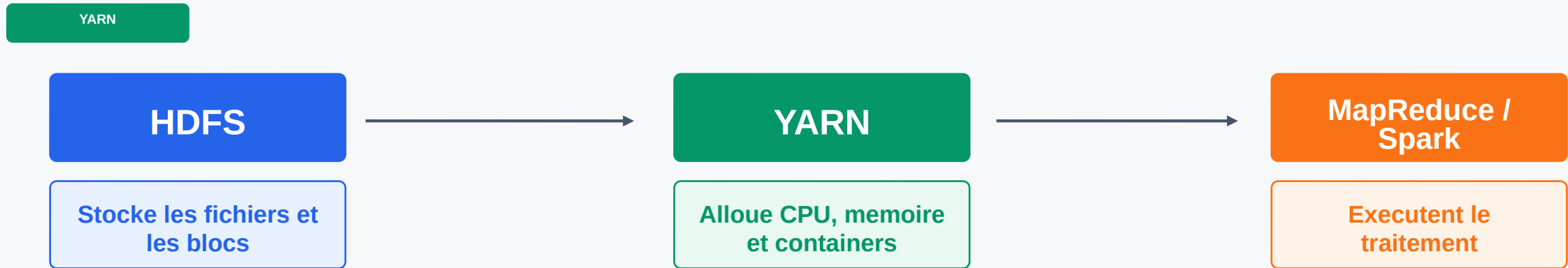
- Copier le fichier orders.log fourni dans HDFS.
- Lister le repertoire HDFS cible.
- Afficher les 5 premieres lignes.
- Calculer la taille et le nombre de fichiers.
- Supprimer puis recreer le dossier de sortie si necessaire.

```
hdfs dfs -mkdir -p /shopflow/input
hdfs dfs -put -f ~/datasets/orders.log /shopflow/input/
hdfs dfs -ls /shopflow/input
hdfs dfs -cat /shopflow/input/orders.log | head -n 5
hdfs dfs -du -h /shopflow/input
hdfs dfs -count /shopflow/input
```

Question de comprehension : pourquoi cette manipulation reste simple pour l'utilisateur alors que le stockage est distribue ?

YARN : le gestionnaire de ressources du cluster

Une fois les données stockées, il faut décider ou lancer le calcul.

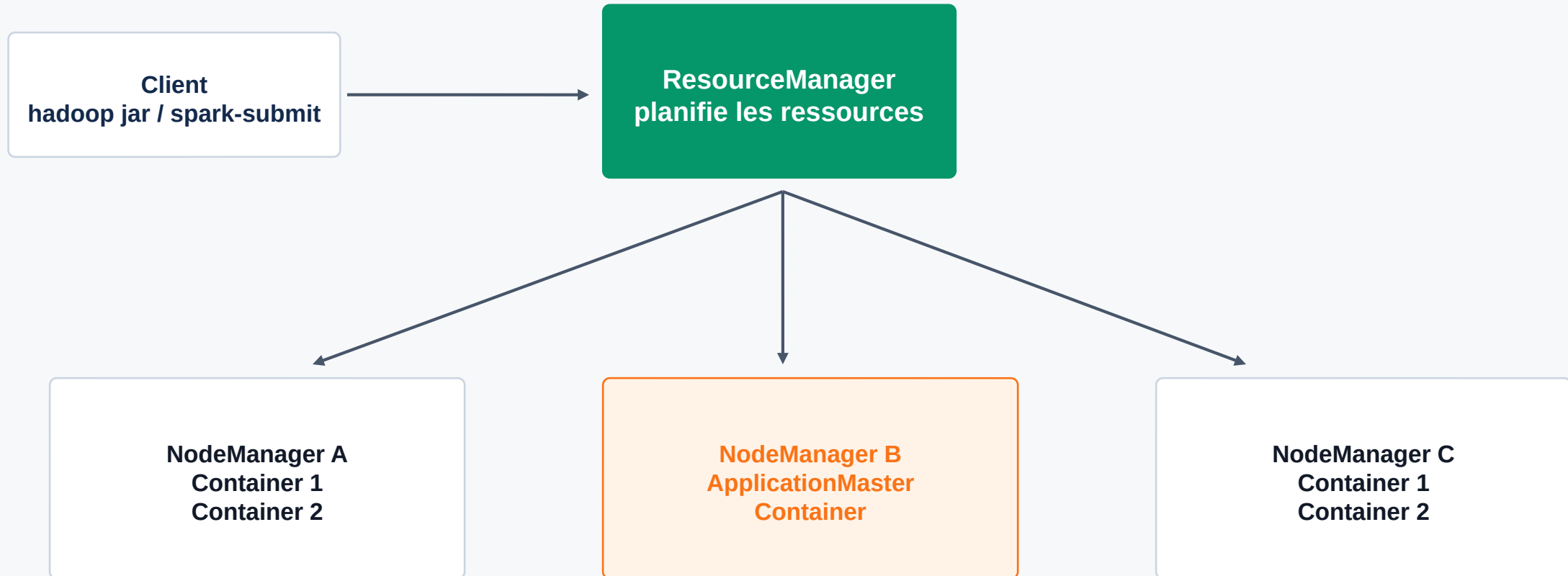


Analogie : HDFS est l'entrepot, YARN est le responsable logistique, MapReduce/Spark sont les équipes de traitement.

Architecture YARN : ResourceManager, NodeManagers, containers

YARN transforme un cluster en reservoir de ressources partageables.

YARN



Container = enveloppe d'execution avec CPU + memoire. Une application peut utiliser plusieurs containers.

Cycle de vie simplifié d'une application YARN

YARN n'exécute pas directement le code : il réserve des containers puis suit l'application.



```
yarn node -list  
yarn application -list  
yarn application -list -appStates ALL
```

Ces commandes permettent de voir les noeuds et les applications connues par YARN.

Exercice 2 - observer YARN sur la VM

Objectif : voir le cluster comme un ensemble de ressources, pas comme une machine locale.

EXERCICE

```
# Voir les noeuds connus par YARN
yarn node -list

# Voir les applications en cours
yarn application -list

# Voir les anciennes applications
yarn application -list -appStates ALL

# Optionnel selon la VM
# Interface YARN : http://localhost:8088
```

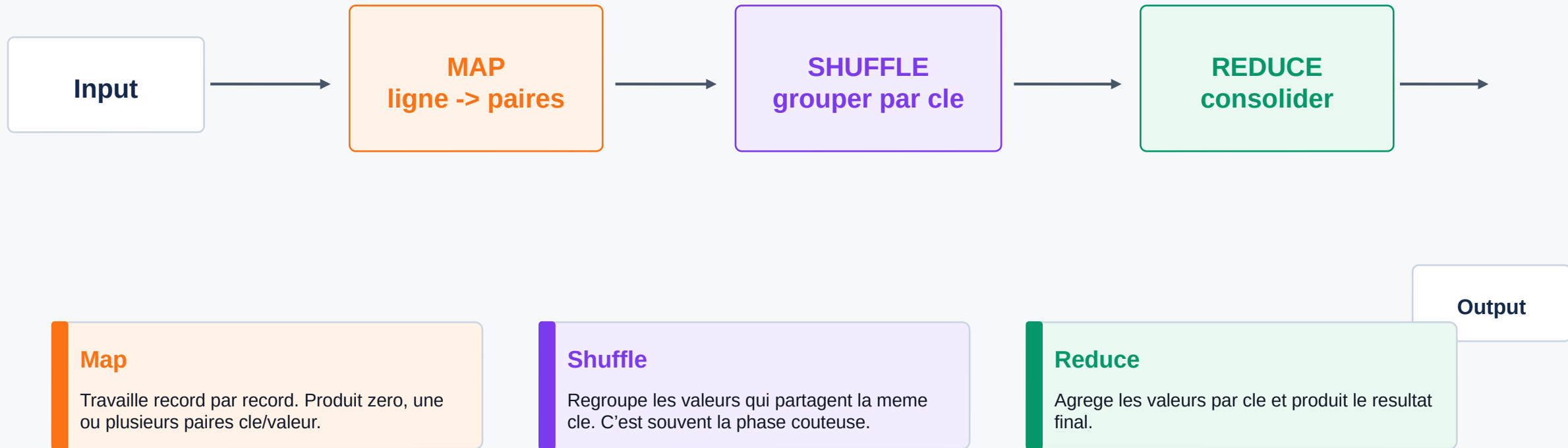
- Combien de noeuds sont visibles ?
- Y a-t-il une application en cours ?
- Que signifie l'état FINISHED ou RUNNING ?
- Quelle différence faites-vous entre un processus local et une application YARN ?

Trace à conserver : une capture de la commande `yarn application -list -appStates ALL` ou de l'interface ResourceManager.

MapReduce : transformer un gros probleme en trois phases

MapReduce impose une discipline simple : Map, Shuffle, Reduce.

MAPREDUCE



Exemple classique : WordCount a la main

Compter des mots permet de voir clairement Map, Shuffle et Reduce.

MAPREDUCE

```
Input
cat dog cat
dog bird
cat bird
```

```
MAP
(cat,1)
(dog,1)
(cat,1)
(dog,1)
(bird,1)
(cat,1)
(bird,1)
```

```
SHUFFLE
cat -> [1,1,1]
dog -> [1,1]
bird -> [1,1]
```

```
REDUCE
cat 3
dog 2
bird 2
```

Le resultat n'est pas produit par un seul gros programme central : plusieurs mappers travaillent en parallele, puis les reducers consolident les cles.

Ce qui se passe dans un job MapReduce

Le code utilisateur est simple, mais l'exécution distribuée est structurée.

MAPREDUCE



- Le fichier d'entrée est découpé en splits.
- Chaque mapper produit des paires cle/valeur.
- Le shuffle trie et regroupe les paires par cle.
- Les reducers écrivent le résultat dans HDFS.
- Le dossier de sortie contient souvent plusieurs fichiers part-*

Important : le dossier de sortie ne doit pas déjà exister, sinon le job échoue.

```
hdfs dfs -rm -r -f /shopflow/output/wordcount
```

Exercice 3 - lancer un WordCount MapReduce

Objectif : executer un vrai job MapReduce sur YARN et lire le resultat dans HDFS.

EXERCICE

```
# 1. Preparer le dossier de sortie
hdfs dfs -rm -r -f /shopflow/output/wordcount

# 2. Trouver le JAR d'exemples Hadoop
EXAMPLES_JAR=$(find /usr -name "hadoop-mapreduce-examples*.jar"
2>/dev/null | head -n 1)
echo $EXAMPLES_JAR

# 3. Lancer WordCount
hadoop jar "$EXAMPLES_JAR" wordcount \
  /shopflow/input/orders.log \
  /shopflow/output/wordcount

# 4. Lire le resultat
hdfs dfs -cat /shopflow/output/wordcount/part-* | sort
```

- Le job est soumis au cluster via Hadoop/YARN.
- Le fichier source est lu depuis HDFS.
- Le resultat est ecrit dans HDFS.
- Le dossier de sortie doit etre absent avant le lancement.
- Pendant ou apres le job, observer YARN.

**Elements de verification : commande executee +
extrait du resultat part-* + application YARN
terminee.**

Exercice 4 - lire le resultat et l'interpreter

Objectif : ne pas seulement lancer un job, mais comprendre ce qui a ete compte.

EXERCICE

```
Extrait orders.log
2026-05-01T10:01:22 orderId=01001 channel=web sku=SKU-USB-C qty=2 status=CREATED
2026-05-01T10:03:11 orderId=01002 channel=mobile sku=SKU-MOUSE qty=1 status=CREATED
2026-05-01T10:04:08 orderId=01003 channel=web sku=SKU-USB-C qty=1 status=CANCELLED
```

- Retrouver dans la sortie le nombre de commandes par channel.
- Retrouver le nombre d'occurrences par sku.
- Retrouver les statuts CREATED, CANCELLED, SHIPPED.
- Expliquer pourquoi WordCount est limite pour faire une vraie analyse de commandes.
- Proposer ce qu'il faudrait faire avec un vrai mapper/reducer dedie.

Synthese : WordCount fonctionne, mais pourquoi n'est-ce pas un vrai moteur analytique confortable ?

Ouverture : Spark / DataFrame permettra de manipuler des colonnes, filtrer, grouper, joindre et agreger plus naturellement.

Exercice 5 - Map / Shuffle / Reduce sur papier

Objectif : savoir decire le traitement sans se cacher derriere la commande.

EXERCICE

Input

```
order001 web CREATED
order002 mobile CREATED
order003 web CANCELLED
order004 partner CREATED
order005 web CREATED
```

Question

- Compter le nombre de commandes par canal.
- Ecrire la sortie de la phase Map.
- Ecrire le regroupement Shuffle.
- Ecrire le resultat Reduce final.

Sortie attendue

```
web      3
mobile   1
partner  1
```

Point important : le choix de la cle determine ce qui peut etre agrege.

Les limites de MapReduce

MapReduce est robuste et scalable, mais il est peu confortable et souvent lent pour certains usages.

LIMITES

Ecritures disque

Chaque job écrit souvent un résultat intermédiaire dans HDFS. Les chaînes de jobs deviennent lentes.

Modele rigide

Tout doit être pensé en Map, Shuffle, Reduce, même si l'analyse serait plus naturelle en SQL ou DataFrame.

Peu interactif

Lancer un job batch, attendre, lire des fichiers part-
* : ce n'est pas idéal pour explorer les données.

Pourquoi c'est important ?

- Les algorithmes itératifs repassent plusieurs fois sur les données.
- Le machine learning et les graphes réutilisent souvent les mêmes datasets.
- L'exploration interactive exige des réponses plus rapides.
- C'est exactement l'espace où Spark apporte beaucoup de valeur.

```
MapReduce : lire -> mapper -> shuffler ->
            reduire -> écrire
Spark      : réutiliser en mémoire quand c'est
            pertinent + API haut niveau
```

Transition vers Spark : ce qui change et ce qui reste

Spark ne supprime pas le besoin de comprendre HDFS et YARN.

COMPARER

Role	MapReduce	Spark
Stockage	HDFS	HDFS, S3, fichiers, bases...
Ressources	YARN	YARN, Kubernetes, Standalone...
Modele	Map -> Shuffle -> Reduce	RDD, DataFrame, SQL, Streaming
Intermediaires	Souvent ecrits sur disque	Memoire si possible, disque si necessaire
Usage	Batch robuste	Batch, SQL, ML, streaming, exploration

Message cle : Spark est plus rapide et plus confortable pour beaucoup d'usages, mais il s'inscrit dans le meme monde distribue : fichiers, partitions, ressources, jobs.

Quiz de consolidation

Verifier les concepts avant de passer au cours Spark.

QUIZ

- 1. Quelle est la difference entre HDFS et YARN ?
- 2. Pourquoi HDFS decoupe-t-il les fichiers en blocs ?
- 3. Que fait le NameNode ? Que fait un DataNode ?
- 4. Dans WordCount, quelle est la cle apres la phase Map ?
- 5. Pourquoi le Shuffle est-il souvent couteux ?
- 6. Pourquoi MapReduce est-il moins confortable que Spark pour l'analyse interactive ?

Temps de consolidation : 3 minutes en binome, puis mise en commun des reponses.

Reperes de correction du quiz

Les reponses de reference sont simples, mais techniquement justes.

CORRECTION

HDFS vs YARN

HDFS stocke les donnees. YARN gere les ressources et lance les applications.

Blocs HDFS

Decouper permet de distribuer, paralleliser la lecture et repliquer pour tolerer les pannes.

NameNode/DataNode

NameNode = carte des fichiers et des blocs.
DataNode = stockage reel des blocs.

Cle WordCount

La cle est le mot ; la valeur est souvent 1.

Shuffle

Couteux car il regroupe les donnees par cle, parfois entre plusieurs machines.

Limite MapReduce

Beaucoup d'ecritures disque, modele rigide, peu adapte a l'exploration et aux algorithmes iteratifs.

Transition vers Spark : maintenant que le socle Hadoop est clair, il devient possible de comprendre ce que Spark garde, simplifie et accelere.

Checklist VM Hadoop/YARN pour l'atelier

Objectif : travailler de façon autonome sur la VM Hadoop/YARN.

VM

```
# Verifier Java et Hadoop
java -version
hadoop version

# Verifier HDFS
hdfs dfs -ls /

# Verifier YARN
yarn node -list
yarn application -list

# Interfaces web possibles selon la version
# NameNode : http://localhost:9870 ou
http://localhost:50070
# YARN RM : http://localhost:8088
```

Fichiers fournis avec la VM

- datasets/orders.log
- datasets/clicks.log
- datasets/catsat.txt
- commands_hdfs_yarn_mapreduce.md
- questions_recap.md

Bon reflexe : toujours verifier ou se trouve un fichier : local Linux ou HDFS.

Ce qu'il faut retenir

Une bonne comprehension de Hadoop rend Spark beaucoup plus lisible.

SYNTHESE

HDFS

Distribue et replique les fichiers. Il repond au probleme du stockage massif.

YARN

Alloue les ressources et pilote le lancement d'applications distribuees.

MapReduce

Modele batch historique : Map, Shuffle, Reduce.
Robuste mais peu flexible.

Comparatif final

- Hadoop apprend a penser stockage et calcul distribues.
- MapReduce impose une logique batch tres structuree.
- Spark reprend l'idee du calcul distribue mais l'exprime avec des API plus haut niveau.
- Pour comprendre Spark sur YARN, il faut deja comprendre HDFS, YARN, partitions, jobs et tasks.

Phrase-cle : Hadoop pose le socle distribue ; Spark rend ce socle plus rapide, plus interactif et plus agreable a programmer.